SYSTEM DESIGN TECHNICAL REFERENCE GUIDE

Access Control Patterns for Distributed Architectures

A Comprehensive Interview Comparative Framework: ACL vs. RBAC vs. ABAC vs. ReBAC

In modern application design, controlling who can perform what action on which resource is a critical security function. For system design interviews, demonstrating a crisp understanding of different access control patterns and their architectural trade-offs is essential to justify production choices. This report synthesizes major authorization models with a focus on core mechanics, database schemas, scaling strategies, and interview-readiness.

1. The Strategic Decision Matrix

Pattern	Primary Use Case	Mental Model	Key Strength	Key Weakness
ACL (Access Control List)	Small-scale systems, file systems, simple network routers.	The **"Guest List"** attached directly to each individual room.	Simple, direct, low conceptual overhead.	Administrative nightmare at scale; no inheritance.
RBAC (Role-Based)	Internal enterprise tools, standard business applications.	The **"Company Badge"** mapping department functions.	Easy to audit, manage, and model statically.	"Role Explosion" under complex or dynamic requirements.
ABAC (Attribute-Based)	Highly regulated environments (e.g., healthcare, finance).	The **"Contextual Guard"** verifying ID, time, and IP.	Infinite flexibility; dynamic and fine-grained.	High evaluation latency; difficult to audit cleanly.
ReBAC (Relationship-Based)	Collaborative SaaS, social graphs, nested hierarchies.	The **"Social Graph"** tracking ownership and sharing paths.	Naturally models user-driven sharing & inheritance.	Graph traversal complexity; consistency lags.

2. Architectural Deep-Dives

1. Access Control Lists (ACL)

The ACL model secures resources by attaching a precise list of permissions directly to each individual resource. It acts as an explicit table stating exactly which users or groups are granted specific actions.

Relational Schema Design:

- `Users` : stores user records (`user_id` , `username`).
- `Resources` : stores securable objects (`resource_id` , `resource_name`).
- `Permissions` : defines valid operations (`permission_id` , `permission_name` , e.g., read, write, execute).

- `ACL`: core join table linking actors to objects (`user_id`, `resource_id`, `permission_id`).

Authorization Query Implementation:

```
SELECT EXISTS (
    SELECT 1 FROM ACL
    WHERE user_id = ? AND resource_id = ? AND permission_id = ?
);
```

Critical Pitfalls: ACLs possess zero native capability for hierarchical inheritance. If a folder's permissions change, an expensive operation must update records for every sub-file to prevent orphaned security states. It degrades rapidly when evaluating long lists or cross-checking complex group hierarchies via multiple SQL joins.

2. Role-Based Access Control (RBAC)

RBAC abstracts access control by mapping permissions directly to a `Role` (e.g., Admin, Editor, Viewer), and then assigning users to one or more of these roles. This cleanly decouples users from explicit action permissions.

The "Role Explosion" Problem: RBAC assumes coarse-grained, static business functions. When application requirements demand granular permissions—such as restricting access by region or specific project—administrators are forced to create specialized roles. This combinatorial explosion can be modeled mathematically as a Cartesian product:

$$Roles_{Total} = Roles_{Job} \times Regions \times Projects$$

For instance, an enterprise with 10 job functions, 5 geographic regions, and 3 active client projects would scale rapidly to $10 \times 5 \times 3 = 150$ highly custom roles (e.g., `Project_Alpha_Manager_NA`), rendering compliance logging and lifecycle management practically unmanageable.

3. Attribute-Based Access Control (ABAC)

ABAC replaces static roles with dynamic, runtime-evaluated policies using contextual metadata. Policies check attributes across four primary classes: **Subject** (e.g., department, clear-level), **Resource** (e.g., owner, classification), **Action** (e.g., read, export), and **Environment** (e.g., current time, IP subnet, system flag).

Standard Policy Evaluation Architecture (PEP, PDP, PIP, PAP):

1. **Policy Enforcement Point (PEP):** The API gateway or middleware that intercepts incoming requests, captures initial context, and awaits an authoritative authorization response.
2. **Policy Decision Point (PDP):** The logic engine that acts as the "brain." It parses the compiled rules and returns an explicit `Permit` or `Deny` statement.
3. **Policy Information Point (PIP):** A critical connector that fetches remote attributes in real-time from external data stores (e.g., querying an HR database or checking a clock).
4. **Policy Administration Point (PAP):** The administrative interface where compliance teams create, modify, and store policies using standardized languages like XACML.

4. Relationship-Based Access Control (ReBAC)

ReBAC derives explicit access rights by traversing a directed graph of entity connections. It is popularized by Google's internal global authorization service, **Zanzibar**. Rather than tracking individual properties, permissions are explicitly derived from the underlying topology of relationships.

The system stores permissions as lightweight **relation tuples** mapping an object to a subject via a designated relationship link:

```
<object>#<relation>@<user_or_userset>
```

Production Tupling Examples:

- `document:doc_q2_report#owner@user:alice` → Alice owns the Q2 report.
- `folder:finance_2026#parent@folder:root_vault` → Nested folder structural hierarchy definition.
- `group:engineering#member@user:bob` → Bob belongs to the engineering team.

ReBAC implements powerful **userset rewrite rules** to execute permission inheritance. For instance, a policy rule can define that an `editor` of a parent folder automatically inherits `viewer` rights across every child document within that graph branch.

3. Distributed Systems Challenges & Mitigations

The ABAC Latency Overhead & Strategic Caching

Evaluating multi-attribute ABAC policies introduces execution latency, as querying external PIPs can degrade endpoint response times. Architects mitigate this overhead via three distinct caching vectors:

- **Policy Caching:** Pre-compiling static XACML rules into fast, in-memory structures to eliminate disk-bound reading during evaluations.
- **Attribute Caching:** Storing dynamic properties (e.g., user roles from active directories) directly inside high-performance key-value caches like Redis with strict Time-To-Live (TTL) policies.
- **Decision Caching (Memoization):** Caching the exact terminal `Permit` / `Deny` answer against a hash of the identical context string to resolve future checks in sub-milliseconds.

The ReBAC "New Enemy Problem"

In highly distributed ReBAC platforms with globally replicated storage nodes (e.g., utilizing Google Spanner), replication delay presents a critical race condition known as the **"New Enemy Problem."**

> **Scenario:** *A coordinator removes a malicious employee's access from a shared folder. Immediately after, an authorized user uploads a highly sensitive file to that same folder. If the secondary authorization node processing the file-view request evaluates against a stale replica that hasn't received the revocation update yet, the malicious user will successfully view the document.*

The Zanzibar Solution: To defeat this race condition without sacrificing performance, the system issues an opaque cryptographic token called a **"Zookie"** whenever permissions change. A Zookie embeds a precise, microsecond-accurate timestamp. Subsequent file-view operations must pass this Zookie along with the request. The

authorization engine forces evaluation to occur only against a snapshot that is at least as fresh as the Zookie timestamp, effectively blocking stale reads.

4. Interview Blueprint: A Framework for Discussion

Step 1: Clarify the Functional Requirements

Avoid prescribing an access control model immediately. Instead, ask targeted architectural scoping questions to define the system's requirements:

- "Are our roles well-defined and corporate-driven, or can end-users dynamically generate and share content paths?"
- "Do we have strict contextual requirements, such as restricting access based on corporate networks, geo-fenced boundaries, or emergency conditions?"
- "Is this a multi-tenant application where clients must isolate their spaces or independently customize user structures?"

Step 2: Propose a High-Performance Hybrid Model

Production distributed platforms rarely implement a single pattern in isolation. Proposing a clean hybrid architecture demonstrates senior-level system architecture insight:

Option A: RBAC + ReBAC (Collaborative SaaS / Multi-Tenant Platforms)

- **Architecture:** Use coarse-grained RBAC to define core cross-platform states (e.g., `Admin`, `Standard User`, `Billing Contact`). Layer a localized ReBAC engine on top to manage fine-grained, nested user-driven sharing.
- **Example:** In a Figma-style workspace, standard global billing is verified quickly via memory-cached RBAC, while granular element viewing paths traverse a highly parallelized ReBAC graph.

Option B: RBAC + ABAC (Regulated Enterprise / Healthcare Environments)

- **Architecture:** Model foundational job responsibilities via traditional RBAC tables. Inject the resulting role as a core subject attribute inside an external ABAC engine to evaluate dynamic variables.
- **Example:** A user with the RBAC role of `Nurse` can access a `MedicalRecord` only if the ABAC engine verifies that $environment.time$ matches their active shift schedule AND $resource.ward == subject.assigned\_ward$.

5. The "Interview Pivot" Script

This script serves as a conceptual blueprint to defend architectural choices during a high-stakes design phase:

> **The Choice:** *"For this system, I am selecting a hybrid* **RBAC + ReBAC** *pattern over a pure RBAC implementation to manage our resource-sharing requirements."*
>
> **Acknowledging the Trade-off:** *"While a native graph-based ReBAC approach introduces operational complexity and requires careful handling of distributed data-replication delays via consistency tokens, it successfully prevents the* **Role Explosion** *pitfall common to traditional relational setups."*
>
> **The Alternatives:** *"If our scaling constraints were strictly confined to internal employees within an isolated corporate network, a simplified relational RBAC model would offer lower operational overhead. However, given our projected multi-tenant load and requirements for user-driven data delegation, implementing an isolated, Zanzibar-inspired relationship engine is necessary to ensure long-term performance and maintainability."*

6. Operational Trade-Off Summary

Constraint Vector	Architectural Winner	Technical Rationale
Highest Read Throughput	**RBAC**	Evaluates flat relational rows with simple index scans; highly cacheable using fast in-memory key-value stores.
Lowest Latency Floor	**ACL / RBAC**	Avoids complex recursive graph traversals or blocking real-time external network API calls.
Context & Risk Adaptation	**ABAC**	The only model that handles real-time environment variables (e.g., blacklisting IPs, detecting anomalous behavior).
Hierarchical Complexity	**ReBAC**	Uses native graph data structures to resolve complex inheritance chains without requiring database alterations.